LOGICA Y REPRESENTACION 1
PARCIAL 2 – ESTRUCTURAS DE CONTROL

Nombre: _____Solución_____ **Identificación**: _____Solución_____

Instrucciones generales

- Los programas de los puntos 2 a 4 se escriben a mano, en Python.
- Use solo lo visto en clase hasta ciclos: variables, operadores, `input` y `print`, condicionales, `while` y `for` con `range`.
- No use listas, métodos de cadena (`.split()`, `.join()`, etc.), multiplicación de cadenas, funciones como `min()` o `max()`, ni librerías (`import`).

1. **(30 %) Análisis de código.**
   Dado el siguiente código en Python (los números de la izquierda son los números de línea y **no** hacen parte del código):

```
 1  i = 10
 2  a = True
 3  j = 100
 4  s = 0
 5  while(i >= 0):
 6      if(i >= 4 and a):
 7          i -= 2
 8          for j in range(i,0,-2):
 9              print(f"{j}", end = " ")
10          print()
11          a = (j == 0)
12      elif(i >= 3):
13          i -= 1
14          continue
15          i -= 2
16          print(i)
17      elif(i >= 1):
18          print(i)
19          a = True
20          i -= 2
21      else:
22          i += 12
23          print("+++++")
24          break
25          print("-----")
26      print("-----")
27      s = s + (i + j)
28  print("Nunca llega")
29  print(i,j,a,s)
```

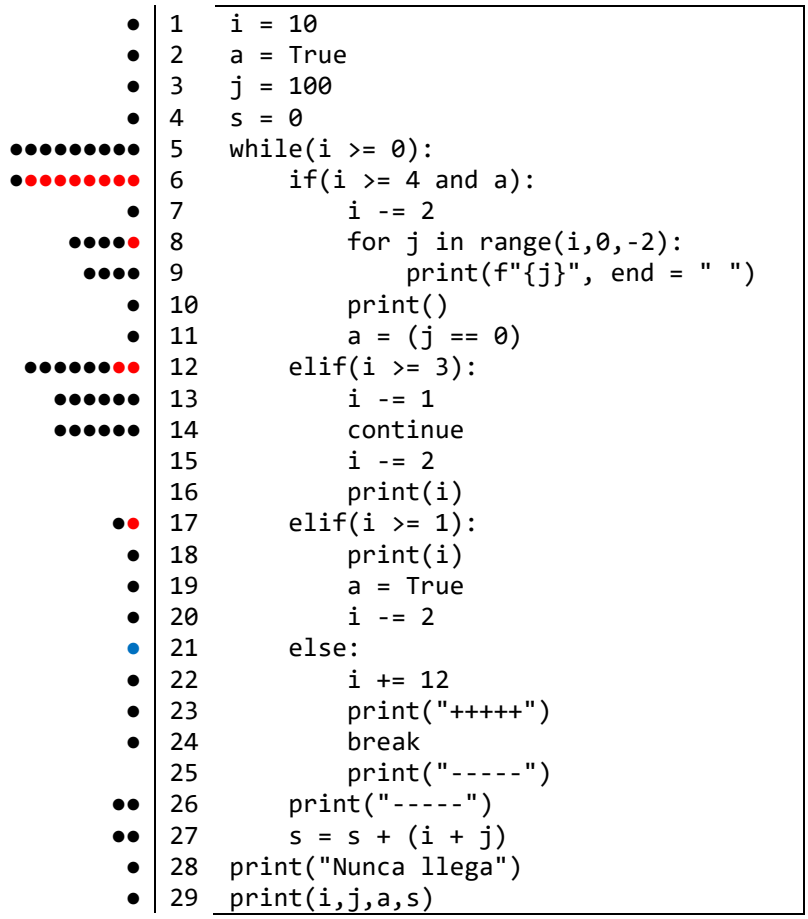
Prueba de escritorio

A continuación, se lleva a cabo la prueba de escritorio del código anteriormente mostrado. Allí se muestra, la evolución de las variables y la salida en pantalla. Así mismo, cada una de las bolitas que aparecen a la izquierda de los números muestra la cantidad de veces que se ejecuto la instrucción. La convención es la siguiente:

- **Bolita negra ()**: Ejecución normal del a instrucción.
- **Bolita roja ()**: Ejecución de una instrucción de control de flujo cuyo resultado es falso.

- **Bolita azul ()**: Para entender la ejecución del programa, si va o no va no hay problema, es decir, si no se señala está bien pues la ejecución cuando todas las condiciones son falsas pasa después del `else`.
- **Ausencia de bolita ( )**: La instrucción dada nunca se ejecutó.

Código



```
1   i = 10
2   a = True
3   j = 100
4   s = 0
5   while(i >= 0):
6       if(i >= 4 and a):
7           i -= 2
8           for j in range(i,0,-2):
9               print(f"{j}", end = " ")
10          print()
11          a = (j == 0)
12      elif(i >= 3):
13          i -= 1
14          continue
15          i -= 2
16          print(i)
17      elif(i >= 1):
18          print(i)
19          a = True
20          i -= 2
21      else:
22          i += 12
23          print("+++++")
24          break
25          print("-----")
26      print("-----")
27      s = s + (i + j)
28  print("Nunca llega")
29  print(i,j,a,s)
```

Prueba de escritorio

i	a	j	s
~~10~~	~~True~~	~~100~~	~~0~~
~~8~~	~~False~~	~~8~~	~~10~~
~~7~~	True	~~6~~	12
~~6~~		~~4~~	
~~5~~		2	
~~4~~			
~~3~~			
~~2~~			
~~0~~			
12			

Salida en pantalla

```
8 6 4 2
-----
2
-----
+++++
Nunca llega
12 2 True 12
```

Responda las siguientes preguntas. Tenga en cuenta que las respuestas deben estar soportadas por una prueba de escritorio, que puede hacer en una hoja adicional a parte (o al lado del código si les queda espacio).

Una vez realizada la prueba de escritorio, responder las siguientes preguntas (tenga en cuenta que la respuesta debe estar correctamente soportada por el proceso llevado a cabo en la prueba de escritorio):

a. **(4 %)** ¿Cuántas veces se repite (cuántas iteraciones hace) el ciclo de la línea 5? __9__

b. **(6 %)** ¿Cuáles son los valores finales de las variables?

Variable	i	j	a	s
Valor	12	2	True	12

c. **(4 %)** ¿Qué imprimen las líneas 8 a 10 la primera vez que se ejecutan?

- ○ 8 6 4 2 0
- ○ 8 8 8 8
- ○ 10 8 6 4 2
- ○ 8 6 4 2

d. **(4 %)** ¿Cuántas veces se ejecuta la línea 13?
   - 5
   - 6
   - 7
   - 8

e. **(4 %)** ¿Qué valor toma la variable a al ejecutar la línea 11, y por qué?
   - `True`, porque el `for` termina cuando `j` llega a `0`
   - `False`, porque al terminar el `for` la variable `j` vale 2
   - `False`, porque `j` vale 8 al empezar el `for`
   - `True`, porque en esa vuelta no se modifica a

f. **(4 %)** ¿Cuáles de las siguientes líneas **nunca** se ejecutan? (Puede seleccionar varias si aplican)
   - Línea 13
   - Línea 15
   - Línea 16
   - Línea 22
   - Línea 25
   - Línea 28

g. **(4 %)** Después de ejecutar completamente el programa, ¿cuáles de las siguientes afirmaciones son correctas? (Puede seleccionar varias si aplican)
   - El texto `-----` se imprime 2 veces.
   - La línea 9 se ejecuta 4 veces.
   - En algún momento la variable `j` toma el valor 0.
   - La línea 26 se ejecuta en la misma vuelta en que se ejecuta la línea 24.
   - La línea 27 se ejecuta 9 veces.
   - Se imprime el texto `Nunca llega`.

2. (20 %) Diseño de algoritmos

Varias ambulancias recorren la ciudad. Cuando la central recibe una llamada, se conoce la ubicación de la emergencia mediante sus coordenadas `(x, y)`, y también la ubicación de cada una de las `N` ambulancias en servicio.

Escriba en Python un programa que lea las coordenadas de la emergencia, la cantidad `N` de ambulancias y las coordenadas de cada una, y que informe las coordenadas de la ambulancia más cercana a la emergencia y la distancia a la que se encuentra.

Tenga en cuenta:

- La distancia entre dos puntos es: $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$
- La raíz se puede calcular con `** 0.5`.
- Suponga que `N >= 1` y que las coordenadas pueden ser números reales.
- Si dos ambulancias están a la misma distancia, informe la primera que se leyó.

Antes del código, llene la siguiente tabla con **todas** las variables que use en su programa.

Solución:

A continuación se describen cada una de las variables empleadas en la solución.

Variable	Significado
`N`	Numero de ambulancias reportadas
`x_emerg`	Coordenada x de la emergencia
`y_emerg`	Coordenada y de la emergencia
`x_amb`	Coordenada x asociada a la ambulancia i-esima
`y_amb`	Coordenada y asociada a la ambulancia i-esima
`x_min`	Coordenada x asociada a la ambulancia más cercana a la emergencia

y_min	Coordenada y asociada a la ambulancia más cercana a la emergencia
i	(**Variable de control**) Variable de control asociada a la i-esima ambulancia
d	Distancia entre la ambulancia i-esima y la emergencia
d_min	Distancia mínima a la emergencia

La tabla anterior sirve para las dos implementaciones relacionadas con la solución:

Código – Forma 1: Se asume la distancia mínima con un valor muy grande que tentativamente no se dará en la realidad. Es la forma mas simple.

```
# Punto 2: Ambulancias (Forma 1)

# Coordenada de la emergencia
x_emerg = float(input("Ingrese la coordenada x de la emergencia: "))
y_emerg = float(input("Ingrese la coordenada y de la emergencia: "))
# Numero de ambulancias
N = int(input("Ingrese el número de ambulancias N: "))
d_min = 999999999 # Distancia minima (inicialmente muy grande)
for i in range(N):
    # Entrada de las coordenadas de la i-esima ambulancia
    x_amb = float(input(f"Coordenada x de la ambulancia {i+1}: "))
    y_amb = float(input(f"Coordenada y de la ambulancia {i+1}: "))
    # Calculo de la distancia
    d = ((x_emerg - x_amb)**2 + (y_emerg - y_amb)**2)**0.5
    # Actualizacion de la distancia minima
    if d < d_min:
        d_min = d
        x_min = x_amb
        y_min = y_amb
print(f"Ambulancia más cercana: ({x_min}, {y_min})")
print(f"Distancia: {d_min}")
```

Código – Forma 2: Dentro del ciclo, se inicializa la distancia mínima mediante el calculo de la emergencia con la primera ambulancia leída, para los demás casos lo que sigue es la actualización de la distancia mínima al comparar con las demás distancias calculadas para las otras ambulancias de tal manera que si una de estas distancias especificas el menor que la que esta actualmente como la distancia mínima, el valor de esta se actualiza.

```
# Punto 2: Ambulancias (Forma 2)

# Coordenada de la emergencia
x_emerg = float(input("Ingrese la coordenada x de la emergencia: "))
y_emerg = float(input("Ingrese la coordenada y de la emergencia: "))
# Numero de ambulancias
N = int(input("Ingrese el número de ambulancias N: "))
for i in range(N):
    # Entrada de las coordenadas de la i-esima ambulancia
    x_amb = float(input(f"Coordenada x de la ambulancia {i+1}: "))
    y_amb = float(input(f"Coordenada y de la ambulancia {i+1}: "))
    # Calculo de la distancia a la emergencia de la i-esima ambulancia
    d = ((x_emerg - x_amb)**2 + (y_emerg - y_amb)**2)**0.5
    if i == 0:
        # Inicializacion de la distancia minima con la primera ambulancia
        d_min = d
```

```
        x_min = x_amb
        y_min = y_amb
    else:
        # Actualizacion de la distancia minima
        if d < d_min:
            d_min = d
            x_min = x_amb
            y_min = y_amb
print(f"Ambulancia más cercana: ({x_min}, {y_min})")
print(f"Distancia: {d_min}")
```

Ejemplo de ejecución (tentativo):

```
Ingrese la coordenada x de la emergencia: 3
Ingrese la coordenada y de la emergencia: 4
Ingrese el número de ambulancias N: 4
Coordenada x de la ambulancia 1: 0
Coordenada y de la ambulancia 1: 0
Coordenada x de la ambulancia 2: 5
Coordenada y de la ambulancia 2: 5
Coordenada x de la ambulancia 3: 2
Coordenada y de la ambulancia 3: 3
Coordenada x de la ambulancia 4: 10
Coordenada y de la ambulancia 4: 1
Ambulancia más cercana: (2.0, 3.0)
Distancia: 1.4142135623730951
```

3. (25 %) Diseño de algoritmos

Hacer un algoritmo que lea un valor de `x` y calcule el valor de los primeros `N` términos de la serie:

$$1 - \frac{x^3}{2*1} + \frac{x^5}{3*4} - \frac{x^7}{5*6} + \frac{x^9}{7*8} - \cdots$$

Tenga en cuenta:

- El programa debe hacerse en Python, leer `x` y `N`, y mostrar la `suma` de los primeros `N` términos.
- El primer término de la serie es el `1`. Suponga que `N >= 1`.

Antes del código, llene la siguiente tabla con **todas** las variables que use en su programa.

Solución:

A continuación, se muestran las variables empleadas en el código y su función:

Variable	**Significado**
`term_1`	Termino menor del denominador
`term_2`	Termino mayor del denominador
`expo`	(**contador**) Exponente del numerador
`suma`	(**acumulador**) Suma de los términos
`signo`	(**bandera**) Signo (True: Positivo – False: Negativo)
`X`	Variable x
`N`	Numero de términos a sumar
`i`	(**Variable de control**) Variable de control del ciclo (asociada termino i-esimo)
`term`	Termino que se suma o resta al acumulador según el signo

Nótese el uso de contadores para facilitar generar los términos de la expresión de tal manera que no se dependa de una única variable.

Código:

```
########## Inicializacion de variables ##########
# Contadores
term_1 = 1   # Termino menor del denominador
term_2 = 2   # Termino mayor del denominador
expo = 3     # Exponente del numerador
# Bandera
signo = False  # True: positivo, False: negativo
# Acumulador
suma = 1 # Resultado de la suma de los terminos

########## Entrada de datos ##########
X = float(input("Ingrese el valor de X: "))
N = int(input("Ingrese el número de términos N: "))

########## Ciclo para el calculo de la suma #########
for i in range(0, N - 1):
    # Calculo del termino
    term = (X**expo)/(term_1*term_2)
    # Alternancia de signos
    if signo:
        suma += term
    else:
        suma -= term
    # Actualizacion de variables
    term_1 += 2
    term_2 += 2
    expo += 2
    signo = not signo # Cambio de signo

####### Impresion del resultado #######
print(f"La suma de los primeros {N} términos es: {suma}")
```

Ejemplo de ejecución (tentativo):

```
Ingrese el valor de X: 1
Ingrese el número de términos N: 3
La suma de los primeros 3 términos es: 0.5833333333333334
```

4. (**25 %**) **Diseño de algoritmos**

Escriba en Python un programa que lea un número entero N (suponga `N >= 1`) e imprima un triángulo hueco de `N` filas como el del ejemplo: la última fila, la última columna y la diagonal están formadas por asteriscos, y el resto de las posiciones son espacios en blanco.

Restricción: use ciclos `for` anidados. Cada carácter, asterisco o espacio, debe imprimirse con su propio `print`.

Antes del código, llene la siguiente tabla con **todas** las variables que use en su programa:

Solución:

A continuación, se muestran las variables empleadas en el código y su función:

Variable	Significado
N	Numero de filas a imprimir
num_espacios1	(**contador**) Numero de espacios afuera del triángulo (izquierda)
num_espacios2	(**contador**) Numero de espacios al interior del triángulo (centro)
i	(**variable de control**) Variable de control del ciclo externo (número de renglones)
j	(**variable de control**) Variable de control del ciclo interno (número de espacios externos)
	(**variable de control**) Variable de control del ciclo interno (número de espacios internos)

Nótese el uso de contadores y variables de control asociadas a los ciclos. Además, note que la variable de control j se usa primero para controlar la cantidad de espacios externos (a la izquierda) y luego para controlar la cantidad de espacios internos (entre los dos asteriscos).

Código:

```
# Entrada de datos
N = int(input("Ingrese el numero de filas: "))
print()  # Cambio de linea

########## Inicializacion de variables ##########
# Contadores
num_espacios1 = N - 1 # Espacios externos
num_espacios2 = 0     # Espacios internos

######### Ciclo para la impresion de las filas #########
for i in range(N):
    # Impresion de los espacios externos
    for j in range(num_espacios1):
        print(" ", end="")
    # Impresion de la parte central de la fila
    if i == 0:
        # Primera fila
        print("*", end="") # Un asterisco
    elif i < N - 1:
        # Filas intermedias
        print("*", end="") # Primer asterisco
        # Espacios internos
        for j in range(num_espacios2):
            print(" ", end="")
        print("*", end="") # Segundo asterisco
        num_espacios2 += 1 # Actualizacion del numero de espacios internos
    else:
        # Ultima fila con N asteriscos
        for j in range(N):
            print("*", end="")
    print()  # Cambio de linea
    num_espacios1 -= 1 # Disminucion del numero de espacios externos
```

Ejemplo de ejecución (tentativo) con `N = 6`:

```
Ingrese el número de filas N: 6

     *
    **
   * *
  *  *
 *   *
******
```